

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering
ASSESSMENT TOOL 3 – Multimodal Mini-Demo (Text-to-Image + Speech-to-Text/Text-to-Speech)

Course Code:	CSA65	Course Title:	Generative AI and Large Scale Models
CO Assessed:	CO4 – Precision (BL3, BL4,BL5)	Assessment:	Assessment Tool 3 - Multimodal Mini-Demo (Text-to-Image + Speech-to-Text/Text-to-Speech)
Weightage:	10% of CIA	Total Marks:	2 * 50= 100
CO4 Statement:	<i>Design and implement multimodal Generative AI applications integrating text, image, and speech models for engineering applications.(BL3, BL4,BL5)</i>		
Student Name:	J KARTHIK	Reg. No.:	192472136
Section / Batch:	CSE – AI / 2024	Date:	

Instructions to Students

- Answer any 2 questions. Each question carries 50 marks. Total: 100 marks.
- Develop and execute the assigned **Python program** using Text-to-Image, Speech-to-Text, or Text-to-Speech.
- Explain the **working principle and program workflow**.
- Demonstrate the program with **suitable inputs and outputs**.
- Use appropriate **libraries/APIs** and include basic **error handling**.
- Show the **execution output/screenshots**.
- For integrated tasks, demonstrate the complete **multimodal workflow**.
- Include a **workflow/architecture diagram or comparison table**.
- Mention a **real-world application** of the implemented solution.
- Explain the **strengths and limitations** of the approach.
- Include proper **references/citations** for the libraries, APIs, models, or research sources used.

Code of Conduct

I J KARTHIK / 192472136 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: J KARTHIK

18. Write a program that saves the recognized speech into a .txt file.

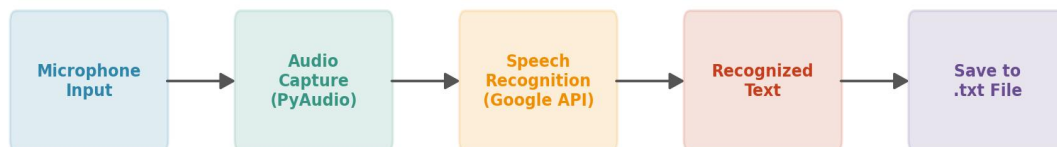
1. Requirement / Objective

- Capture live speech from the microphone.
- Convert the captured speech into text using a Speech-to-Text (STT) model/API.
- Save the recognized text permanently into a .txt file on disk.
- Handle errors gracefully (no speech detected, unclear audio, network/service failure, missing microphone).

2. Working Principle & Program Workflow

The program uses the SpeechRecognition Python library, which acts as a wrapper around several speech engines — here the free Google Web Speech API is used. The microphone audio stream is first captured as raw audio; the recognizer then calibrates itself against ambient/background noise so that silence is not misread as speech. The captured audio segment is sent to the recognition engine, which returns the most probable text transcription. This text is then written to a .txt file using standard Python file-handling (open/write), making the result persistent even after the program closes.

Q18: Speech-to-Text → Save Recognized Speech to .txt File



3. Python Program

Q18: Speech-to-Text Program that saves recognized speech into a .txt file

Library used: SpeechRecognition (Google Web Speech API)

```
"""
```

```
import speech_recognition as sr
```

```
def recognize_and_save(output_file="recognized_speech.txt"):
    recognizer = sr.Recognizer()
```

```
    try:
        with sr.Microphone() as source:
```

```

print("Adjusting for ambient noise... please wait.")
recognizer.adjust_for_ambient_noise(source, duration=1)
print("Listening... Speak now.")
audio = recognizer.listen(source, timeout=5, phrase_time_limit=10)

print("Recognizing speech...")
recognized_text = recognizer.recognize_google(audio)
print("You said:", recognized_text)

# Save recognized text into a .txt file
with open(output_file, "w", encoding="utf-8") as f:
    f.write(recognized_text)
print(f'Recognized speech successfully saved to '{output_file}'')

except sr.WaitTimeoutError:
    print("Error: No speech detected within the timeout period.")
except sr.UnknownValueError:
    print("Error: Could not understand the audio. Please speak clearly.")
except sr.RequestError as e:
    print(f'Error: Could not request results from the speech service; {e}')
except OSError:
    print("Error: No microphone was detected on this system.")

if __name__ == "__main__":
    recognize_and_save()

```

4. Libraries / APIs Used

- **SpeechRecognition (pip install SpeechRecognition)** – interfaces with multiple STT engines.
- **PyAudio (pip install PyAudio)** – required by SpeechRecognition to access the system microphone.
- **Google Web Speech API** – free, key-less cloud engine used via `recognize_google()`.

5. Sample Input & Output

Input (spoken into the microphone):

"Artificial intelligence is transforming modern education."

Console Output:

Adjusting for ambient noise... please wait.

Listening... Speak now.

Recognizing speech...

You said: artificial intelligence is transforming modern education

Recognized speech successfully saved to 'recognized_speech.txt'

Contents written to `recognized_speech.txt`:

artificial intelligence is transforming modern education

Error-handling example (no microphone detected):

Error: No microphone was detected on this system.

6. Real-World Application

This program mirrors the core mechanism behind voice-note transcription tools, meeting-minutes generators (e.g., automatic transcription in Google Meet/Zoom), and voice-based data entry systems used in call centres, healthcare (doctor's dictation), and courtrooms, where spoken statements must be archived as searchable, editable text files.

7. Strengths and Limitations

Strengths	Limitations
Free to use – no API key required for Google Web Speech API.	Requires an active internet connection (cloud-based recognition).
Simple, readable code with minimal setup.	Accuracy drops with background noise, accents, or overlapping speech.
Persistent output – text is saved for later reference/auditing.	Google's free endpoint is rate-limited and not intended for production scale.
Robust exception handling prevents abrupt crashes.	Only transcribes one phrase per run; no continuous/streaming capture.

8. References / Citations

- SpeechRecognition Library Documentation – <https://pypi.org/project/SpeechRecognition/>
- Google Cloud Speech-to-Text / Web Speech API – <https://cloud.google.com/speech-to-text>
- PyAudio Documentation – <https://people.csail.mit.edu/hubert/pyaudio/>
- Python Software Foundation – Python File I/O Documentation – <https://docs.python.org/3/tutorial/inputoutput.html>

32. Write a program that accepts speech → converts speech to text → displays the recognized text.

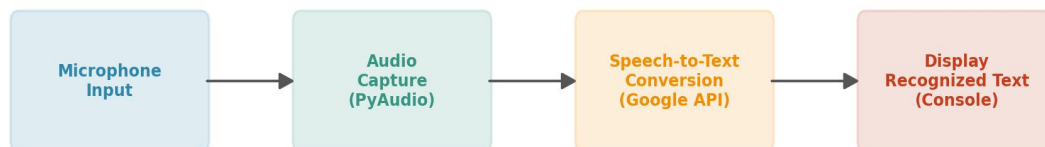
1. Requirement / Objective

- Accept live speech input from the user through the microphone.
- Convert the captured speech into text (Speech-to-Text conversion) using an AI-based recognition engine.
- Display the recognized text clearly on the screen/console.
- This is an Integrated Multimodal Task — it demonstrates the complete audio → text pipeline end-to-end with error handling.

2. Working Principle & Program Workflow

This is an integrated multimodal pipeline that chains two stages together: (1) Audio Acquisition — the microphone stream is opened and background noise is calibrated so the recognizer can isolate the spoken segment; (2) Speech-to-Text Conversion & Display — the isolated audio is sent to the Google Web Speech API, which returns a text transcription; this string is then printed to the console in a clearly formatted block. Unlike Q18, the recognized text here is not persisted to disk — the deliverable of this task is on-screen display, completing the Speech → Text → Display workflow.

Q32: Integrated Task — Speech → Text → Display on Screen



3. Python Program

Pipeline: Speech --> Convert Speech to Text --> Display Recognized Text

Library used: SpeechRecognition (Google Web Speech API)

```
import speech_recognition as sr

def speech_to_text_display():
    recognizer = sr.Recognizer()

    try:
        with sr.Microphone() as source:
```

```

    print("Calibrating microphone for background noise...")
    recognizer.adjust_for_ambient_noise(source, duration=1)
    print("Please speak into the microphone now...")
    audio_data = recognizer.listen(source, timeout=5, phrase_time_limit=10)

    print("Converting speech to text...")
    recognized_text = recognizer.recognize_google(audio_data)

    # Display the recognized text on screen
    print("\n----- Recognized Text -----")
    print(recognized_text)
    print("-----")

    return recognized_text

except sr.WaitTimeoutError:
    print("Error: No speech was detected within the given time.")
except sr.UnknownValueError:
    print("Error: Speech was unclear and could not be recognized.")
except sr.RequestError as e:
    print(f'Error: Could not connect to the speech recognition service; {e}')
except OSError:
    print("Error: No microphone found. Please connect a microphone.")

if __name__ == "__main__":
    speech_to_text_display()

```

4. Libraries / APIs Used

- **SpeechRecognition (pip install SpeechRecognition)** – handles audio capture + STT conversion.
- **PyAudio (pip install PyAudio)** – provides microphone access as the audio backend.
- **Google Web Speech API** – cloud speech-recognition engine invoked via `recognize_google()`.

5. Sample Input & Output

Input (spoken into the microphone):

"Generative AI can create text, images, audio, and other forms of content."

Console Output:

Calibrating microphone for background noise...

Please speak into the microphone now...

Converting speech to text...

----- Recognized Text -----

generative AI can create text images audio and other forms of content

Error-handling example (unclear speech):

Error: Speech was unclear and could not be recognized.

6. Complete Multimodal Workflow

As an integrated task, the program demonstrates the full chain of modalities working together rather than in isolation:

- **Stage 1** – Audio Modality: raw sound waves are captured from the microphone hardware.
- **Stage 2** – Signal Processing: ambient-noise calibration filters the raw signal before recognition.
- **Stage 3** – AI Inference: the Google Web Speech API model converts the processed audio into natural-language text.
- **Stage 4** – Text Modality Output: the transcribed text is rendered on-screen, completing the audio-to-text transformation.

7. Real-World Application

This pipeline is the foundation of voice assistants (Google Assistant, Siri, Alexa), live-captioning systems for accessibility (subtitles for the hearing-impaired), and hands-free command interfaces in cars and smart devices, where spoken input must be instantly converted to readable, actionable text.

8. Strengths and Limitations

Strengths	Limitations
Demonstrates a complete, working multimodal pipeline (speech → text).	Fully dependent on internet connectivity for cloud recognition.
Real-time feedback — recognized text is displayed immediately.	No confidence score or punctuation is returned by the free API.

Strengths	Limitations
Minimal, easy-to-extend code (can be chained with TTS or image generation).	Struggles with domain-specific terms, code-mixed speech, or heavy accents.
Graceful degradation via targeted exception handling.	Single-utterance capture only; not a continuous/streaming listener.

9. References / Citations

- SpeechRecognition Library Documentation – <https://pypi.org/project/SpeechRecognition/>
- Google Cloud Speech-to-Text / Web Speech API – <https://cloud.google.com/speech-to-text>
- PyAudio Documentation – <https://people.csail.mit.edu/hubert/pyaudio/>
- Real Python – 'The Ultimate Guide to Speech Recognition with Python' – <https://realpython.com/python-speech-recognition/>

RUBRICS FOR CALCULATION

Criteria	Wt.	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Pipeline Implementation	30%	Correctly implements a working text-to-image AND speech-to-text/text-to-speech pipeline	Both implemented with minor issues	Only one modality implemented correctly	Neither modality functions correctly
Output Quality & Relevance	25%	Generated outputs are high-quality and clearly relevant to inputs	Outputs are mostly relevant	Outputs are of inconsistent quality	Outputs are largely irrelevant or poor quality
Input Robustness	20%	Handles varied inputs robustly (different prompts/audio clips)	Handles common inputs well	Handles only the simplest inputs	Fails on basic inputs
Demo Polish	15%	Polished, engaging demo of both modalities	Clear demo with minor polish issues	Basic or rough demo	No usable demo presented
Tool/Model Explanation	10%	Clear explanation of models/tools used and their limitations	Adequate explanation	Minimal explanation	No explanation provided

Marks Evaluation:

Question No.	Pipeline Implementation (30%)	Output Quality & Relevance (25%)	Input Robustness (20%)	Demo Polish (15%)	Tool/Model Explanation (10%)	Total Marks (100)
Q1						
Q2						
Overall Total (100)						